

Choose the change you want to make

Start with a task chart, follow a card to its action and rationale, then follow Read in PDF to the captured implementation. NEW marks proposed additions; REUSE marks existing support. The runtime example describes existing behavior.

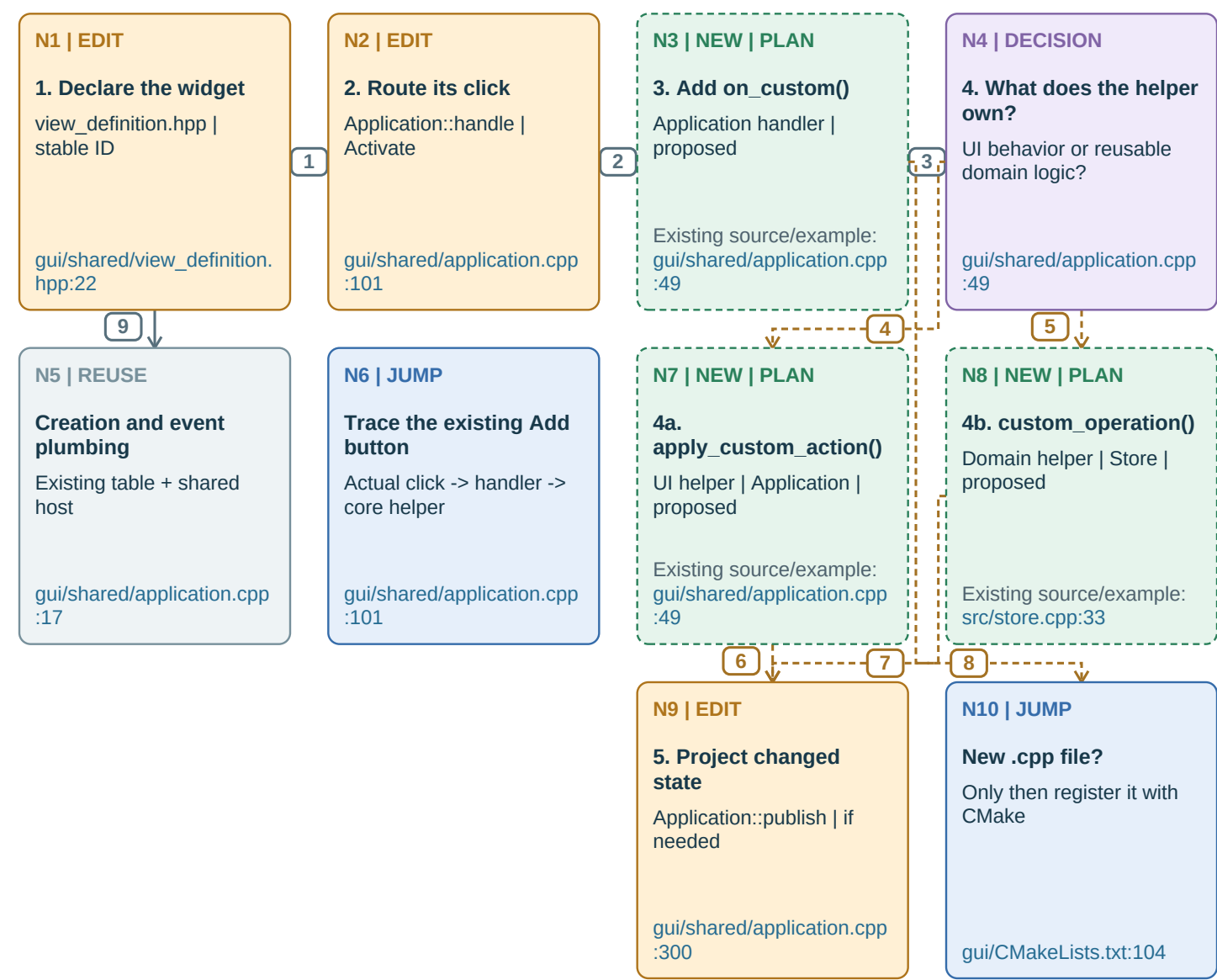
This is a manually generated navigation snapshot. Check current source before editing; the documentation may be stale.

Edit path	Question it answers
Add a clickable widget and a helper	Which files do I edit for a widget, its click event, and a function called beneath that event?
Place a widget in the right position and page	Where do I set its order, height, parent group, and tab?
Add a page tab and place its contents	Which edits create a tab, put widgets beneath it, and make switching work?
Make a new source file compile	Where do I put a new file, and which compiler source list must I edit?
Change record validation or a business rule	Where do I change which records are accepted, and how do errors reach the user?
Edit the selected entry	How do I replace the selected record text while preserving its identity?
Add a menu command or keyboard shortcut	How do I add an action to the menu and reuse its behavior from a shortcut?
Change background work, cancel and progress	Where do I change the work and its displayed result while retaining cancellation and responsiveness?
Change import/export content format	Where do I change record serialization without duplicating file transport across hosts?
Existing example: Add entry	Which existing functions does an Add-entry click reach?

Printable reference and execution charts: [Code walkthroughs: ownership, control, and full source](#) | [Compiler and configuration reference](#) | [What runs when: calls, commands, and event handoffs](#) | [Inside the steps: actual code and configuration flowcharts](#) | [AI-authored: the GUI mental model](#)

[Parameter reference: synopsis, named fields, and filled examples](#)

Which files do I edit for a widget, its click event, and a function called beneath that event?



DIRECTED EDGE KEY

Arrows show edit dependencies. They are not execution arrows. EDIT/NEW boxes identify work; REUSE boxes identify machinery already in place. The existing Add-entry runtime has its own linked map.

1. N1 -> N2
same widget ID
2. N2 -> N3
calls on_custom()
3. N3 -> N4
calls a helper
4. N4 -> N7
UI-only
5. N4 -> N8
domain behavior
6. N7 -> N9
if visible state changes
7. N8 -> N9
if visible state changes
8. N3 -> N10
if any new .cpp
9. N1 -> N5
already consumed

Actions behind this path

Start with the top row. Follow the helper-ownership branch, then use the conditional steps below. Click any box for the edit, the reason, and the exact existing example. `on_custom()`, `apply_custom_action()`, and `custom_operation()` below are proposed names, not existing functions.

[Up to chart](#) | The path is an editing plan. Runtime arrows are identified in the edge key.

N1 | EDIT | 1. Declare the widget

Action: Add a `ViewDefinition` row for a supported widget kind, for example a button with ID entries `custom`, `label`, and `height`. Insert the row at the desired vertical position. For its parent, `tab`, `spacing`, or `custom bounds`, follow the `Place a widget map` linked below. Reuse that exact ID in the event route. A new widget instance uses the existing `gui::Activate` event.

Why here: This ordered table already supplies widget creation, ordering, and shared layout. A new rendering capability or new event type would be a separate contract change.

Source: [gui/shared/view_definition.hpp:22](#) | [Read in PDF \(p. 119\)](#)

[Drill down to the related edit map](#)

Parameters and filled examples: `ViewDefinition`: the widget row, field by field

[Up to chart](#) | [Explore this step](#) | widget-row

N2 | EDIT | 2. Route its click

Action: Inside the `gui::Activate` branch, add a `target-ID` case that calls `on_custom()`, following the `entries.add` case. Keep the existing `close/shutdown` checks and event normalization ahead of dispatch.

Why here: Widget identity selects application behavior. The generic host already forwards events to handle.

Source: [gui/shared/application.cpp:101](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

[Up to chart](#) | [Explore this step](#) | activation

N3 | NEW | 3. Add `on_custom()`

Planned addition to this application. It is not implemented yet; links show existing source and examples.

Action: Declare `on_custom()` in `Application` (normally private), and implement it in `application.cpp`. Have it read the relevant widget state and call your helper. Follow `append_entry()` for owned core calls, status updates, and exception handling; preserve `input_epoch` invalidation when changing authoritative input meaning.

Why here: Separate event dispatch from feature behavior, so another event can reuse the same action.

Existing source/example: [gui/shared/application.cpp:49](#) | [Read in PDF \(p. 11\)](#)

Source: [gui/shared/application.hpp:59](#) | [symbol](#) | [Read in PDF \(p. 14\)](#)

[Up to chart](#) | [Explore this step](#) | handler

N4 | DECISION | 4. What does the helper own?

Action: Choose the helper by responsibility. Application commands and view state stay in the shared GUI. Behavior useful to the CLI and GUI belongs in the core. The helper below the handler does not belong in a renderer.

Why here: This is the decision that determines the next file to edit.

Source: [gui/shared/application.cpp:49](#) | [symbol](#) | [Read in PDF \(p. 11\)](#)

Source: [include/foundation/store.hpp:22](#) | [symbol](#) | [Read in PDF \(p. 18\)](#)

[Up to chart](#) | [Explore this step](#) | helper-owner

N5 | REUSE | Creation and event plumbing

Action: For another instance of an already supported widget, the constructor already creates table rows and the host already forwards events. Inspect these anchors if you need context.

Why here: Ordinary application widget behavior is shared across backends.

Source: [gui/shared/application.cpp:17](#) | [symbol](#) | [Read in PDF \(p. 20\)](#)

Source: [gui/host/contract.hpp:46](#) | [symbol](#) | [Read in PDF \(p. 22\)](#)

[Up to chart](#) | [Explore this step](#) | existing-machinery

N6 | JUMP | Trace the existing Add button

Action: Open the existing Add-entry runtime map to see entries.add -> handle -> append_entry -> Store::add -> Store::validate, followed by publication from handle.

Why here: Use a concrete, source-linked implementation as the pattern for the proposed edit.

Source: [gui/shared/application.cpp:101](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:49](#) | [symbol](#) | [Read in PDF \(p. 11\)](#)

Source: [src/store.cpp:33](#) | [symbol](#) | [Read in PDF \(p. 23\)](#)

[Drill down to the related edit map](#)

[Up to chart](#) | [Explore this step](#) | [existing-example](#)

N7 | NEW | 4a. apply_custom_action()

Planned addition to this application. It is not implemented yet; links show existing source and examples.

Action: Implement apply_custom_action() beside the other Application methods and declare it in application.hpp if it is a member. The proposed on_custom() calls it. A file-local free helper may stay in the same .cpp without adding a header declaration.

Why here: Feature state and command meaning belong here. Existing .cpp files are already compiled.

Existing source/example: [gui/shared/application.cpp:49](#) | [Read in PDF \(p. 11\)](#)

Source: [gui/shared/application.hpp:59](#) | [symbol](#) | [Read in PDF \(p. 14\)](#)

[Up to chart](#) | [Explore this step](#) | [gui-helper](#)

N8 | NEW | 4b. custom_operation()

Planned addition to this application. It is not implemented yet; links show existing source and examples.

Action: Add reusable behavior to the appropriate core module. For the proposed Store::custom_operation(), declare the API in store.hpp and implement it in store.cpp; then call it from the Application handler. The existing append_entry() -> Store::add() relationship is the pattern.

Why here: Core behavior should be usable without a GUI adapter or widget ID.

Existing source/example: [src/store.cpp:33](#) | [Read in PDF \(p. 23\)](#)

Source: [include/foundation/store.hpp:22](#) | [symbol](#) | [Read in PDF \(p. 18\)](#)

Source: [gui/shared/application.cpp:49](#) | [symbol](#) | [Read in PDF \(p. 11\)](#)

[Up to chart](#) | [Explore this step](#) | [core-helper](#)

N9 | EDIT | 5. Project changed state

Action: If the widget needs derived text, enabled state, status, or custom bounds, update publish(). Table-driven creation and normal vertical layout already apply. handle() calls publish() after dispatch returns; do not assume append_entry() calls it.

Why here: Presentation is a projection of shared state. adapter_.present receives the resulting snapshot.

Source: [gui/shared/application.cpp:300](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [gui/shared/application.cpp:163](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:357](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

[Up to chart](#) | [Explore this step](#) | [projection](#)

N10 | JUMP | New .cpp file?

Action: If all new functions remain in an already compiled .cpp, no source-list addition is needed. If you create a new .cpp, open the source-file map and register it with its owning target.

Why here: A new function and a new translation unit are different extension steps.

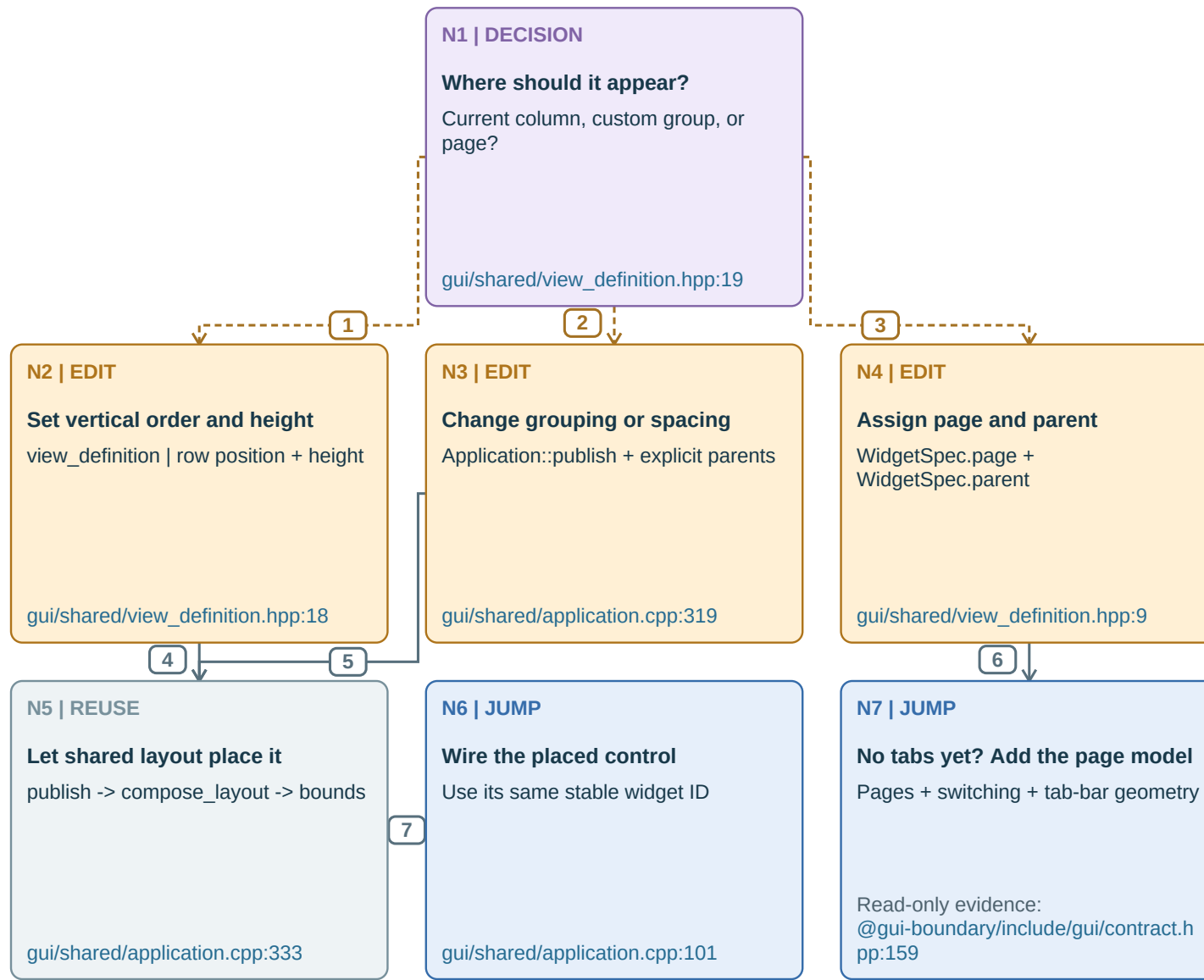
Source: [gui/CMakeLists.txt:104](#) | [Read in PDF \(p. 7\)](#)

Source: [CMakeLists.txt:58](#) | [Read in PDF \(p. 4\)](#)

[Drill down to the related edit map](#)

[Up to chart](#) | [Explore this step](#) | [new-translation-unit](#)

Where do I set its order, height, parent group, and tab?



DIRECTED EDGE KEY

Arrows choose placement edits. Ordinary ordering reuses the current column; custom grouping and pages require shared declaration and layout changes. These are editing paths, not runtime calls.

1. N1 -> N2
ordinary column
2. N1 -> N3
rows / groups
3. N1 -> N4
page membership
4. N2 -> N5
computed placement
5. N3 -> N5
matching layout tree
6. N4 -> N7
first establish pages
7. N5 -> N6
then behavior

Actions behind this path

Today this application has one vertical panel and no page tabs. In that panel, table order sets vertical order and each row sets height. An ID prefix does not select a tab. For page membership, first follow the Add a tab path; the current declaration model has no page or parent field.

[Up to chart](#) | The path is an editing plan. Runtime arrows are identified in the edge key.

N1 | DECISION | Where should it appear?

Action: Choose the existing entries.form column, a custom arrangement, or a page tab. The current app has no tabs. Do not infer parent or page from an ID such as settings.button.

Why here: The placement choice determines whether a row edit is enough.

Source: [gui/shared/view_definition.hpp:19](#) | [Read in PDF \(p. 119\)](#)

Source: [gui/shared/application.cpp:33](#) | [symbol](#) | [Read in PDF \(p. 31\)](#)

[Up to chart](#) | [Explore this step](#) | [placement-choice](#)

N2 | EDIT | Set vertical order and height

Action: Insert or move the widget row between the controls that should appear above and below it. Set its height field. Keep entries.form first: add() currently parents every other widget to that root. The normal publish() loop consumes the rows in this order; optional remove_extension rows appear only when enabled.

Why here: A row in this table is the ordinary vertical placement edit; coordinates are computed later.

Source: [gui/shared/view_definition.hpp:18](#) | [Read in PDF \(p. 119\)](#)

Source: [gui/shared/view_definition.hpp:22](#) | [Read in PDF \(p. 119\)](#)

Source: [gui/shared/application.cpp:330](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Parameters and filled examples: ViewDefinition: the widget row, field by field

[Up to chart](#) | [Explore this step](#) | [table-order](#)

N3 | EDIT | Change grouping or spacing

Action: For spacing, edit panel.gap and panel.padding in publish(). For horizontal rows or nested groups, build the corresponding shared LayoutNode tree and extend ViewDefinition/add() to assign matching explicit parents. Each parent group must precede its children in the snapshot. Update the existing entries.form clipping assumptions when introducing additional root groups; no renderer should position application-specific controls.

Why here: The layout tree computes bounds; WidgetSpec.parent supplies containment and clipping. Both must agree.

Source: [gui/shared/application.cpp:319](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [gui/shared/application.cpp:320](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [gui/shared/application.cpp:321](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [gui/shared/application.cpp:33](#) | [symbol](#) | [Read in PDF \(p. 31\)](#)

Parameters and filled examples: gui::Rect: left, top, width, and height

[Up to chart](#) | [Explore this step](#) | [custom-layout](#)

N4 | EDIT | Assign page and parent

Action: Once pages are initialized, set the intended page ID explicitly on the widget and on its parent group. Give each page its own root group before its children, and map the declaration fields in add(). An empty page means shared chrome. A nonempty page must name a declared page; parent and child must use the same page.

Why here: Names such as entries.custom are widget IDs only; they do not establish tab membership.

Source: [gui/shared/view_definition.hpp:9](#) | [symbol](#) | [Read in PDF \(p. 4\)](#)

Source: [gui/shared/application.cpp:33](#) | [symbol](#) | [Read in PDF \(p. 31\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:82](#) | [symbol](#) | [Read in PDF \(p. 41\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:266](#) | [symbol](#) | [Read in PDF \(p. 43\)](#)

Parameters and filled examples: Widget membership: name the page and parent explicitly

[Up to chart](#) | [Explore this step](#) | [choose-page](#)

N5 | REUSE | Let shared layout place it

Action: Keep layout in `publish()`: compose within the client area, then copy flattened layout boxes into `widget.state.bounds`. The present app uses a 16-unit outer inset and a column with 8-unit padding and 4-unit gaps. Use per-page content bounds when a tab bar is introduced. Resize events reach the same publication path.

Why here: Changing a backend-specific pixel position would bypass the shared placement source.

Source: [gui/shared/application.cpp:333](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [gui/shared/application.cpp:341](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [gui/shared/application.cpp:300](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Parameters and filled examples: `gui::Rect`: left, top, width, and height

[Up to chart](#) | [Explore this step](#) | [compose-bounds](#)

N6 | JUMP | Wire the placed control

Action: After placement, follow the widget/event/helper path to implement the control behavior. Adding only a row or member method in an existing .cpp does not add a compiler source-list step.

Why here: Placement and click behavior are distinct edits, connected by the widget ID.

Source: [gui/shared/application.cpp:101](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

[Drill down to the related edit map](#)

[Up to chart](#) | [Explore this step](#) | [placement-event](#)

N7 | JUMP | No tabs yet? Add the page model

Action: Follow Add a tab for the first page system and for subsequent pages. It covers page declarations, membership, layout, tab-bar bounds, `PageEvent` routing, and active-page/focus consistency.

Why here: Another `ViewDefinition` row alone cannot create a tab in this application.

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:159](#) | [symbol](#) | [Read in PDF \(p. 52\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:135](#) | [symbol](#) | [Read in PDF \(p. 54\)](#)

[Drill down to the related edit map](#)

[Up to chart](#) | [Explore this step](#) | [create-pages](#)

Add a page tab and place its contents

Which edits create a tab, put widgets beneath it, and make switching work?



DIRECTED EDGE KEY

Arrows show the required application edit sequence. All edits belong in gui/shared; supported tabs and controls reuse generic adapters. NEW marks application state to add, not existing functionality.

- 1. N1 -> N2**
page IDs
- 2. N2 -> N3**
explicit ownership
- 3. N3 -> N4**
matching roots
- 4. N4 -> N5**
visible navigation
- 5. N5 -> N6**
publish
- 6. N6 -> N7**
presented selection
- 7. N7 -> N8**
extend content

 EDIT☐ REUSE

 DECISION

 NEW

 JUMP

Click a card for the edit action

Actions behind this path

This is a proposed extension: the current Application initializes no pages. The retained GUI contract already supports Page, WidgetSpec.page, PageEvent, and page_bar; there is no Kind::tabs widget. For the first tabs, complete this path. Once shared support exists, another tab mainly adds a page entry and its owned content. Supplier links are read-only excerpts from the pinned local archive, not files to edit.

[Up to chart](#) | The path is an editing plan. Runtime arrows are identified in the edge key.

N1 | NEW | 1. Declare pages and selection

Planned addition to this application. It is not implemented yet; links show existing source and examples.

Action: Near view_.title in the constructor, initialize view_.pages with stable IDs and labels, for example entries and settings, then set view_.active_page to an available page ID. Keep page IDs unique and nonempty. After this support is implemented, adding another tab starts by adding its Page entry here.

Why here: The tab registry and selection belong to the shared Snapshot. This app currently initializes neither.

Existing source/example: [gui/shared/application.cpp:15](#) | [Read in PDF \(p. 20\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:113](#) | [symbol](#) | [Read in PDF \(p. 56\)](#)

Read-only supplier evidence: [@gui-boundary/examples/application.hpp:14](#) | [symbol](#) | [Read in PDF \(p. 57\)](#)

Parameters and filled examples: [gui::Page](#): a tab ID, caption, and availability

[Up to chart](#) | [Explore this step](#) | [declare-pages](#)

N2 | EDIT | 2. Declare page-owned content

Action: Extend ViewDefinition with explicit page ID and parent ID fields, then update its rows. Declare an entries root group and a settings root group before their children; assign each root and all of its descendants the same page ID. Existing entries controls must be assigned deliberately instead of remaining page-less. Leave page empty only for chrome intentionally shared by all pages.

Why here: The current row schema has no page or parent field; an ID prefix cannot select a tab.

Source: [gui/shared/view_definition.hpp:9](#) | [symbol](#) | [Read in PDF \(p. 4\)](#)

Source: [gui/shared/view_definition.hpp:18](#) | [Read in PDF \(p. 119\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:82](#) | [symbol](#) | [Read in PDF \(p. 41\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:266](#) | [symbol](#) | [Read in PDF \(p. 43\)](#)

Parameters and filled examples: [ViewDefinition](#): the widget row, field by field | [Widget membership](#): name the page and parent explicitly

[Up to chart](#) | [Explore this step](#) | [declare-page-content](#)

N3 | EDIT | 3. Create widgets on their page

Action: Replace add()'s universal first-row parent assignment with the explicit parent and page from each declaration. Preserve unique widget IDs and ensure each parent is an earlier group on the same page. Update any other feature-created widgets to follow that ownership model.

Why here: Without this edit, every widget still belongs to entries.form, regardless of the intended tab.

Source: [gui/shared/application.cpp:33](#) | [symbol](#) | [Read in PDF \(p. 31\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:82](#) | [symbol](#) | [Read in PDF \(p. 41\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:266](#) | [symbol](#) | [Read in PDF \(p. 43\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:265](#) | [symbol](#) | [Read in PDF \(p. 43\)](#)

Parameters and filled examples: [Widget membership](#): name the page and parent explicitly

[Up to chart](#) | [Explore this step](#) | [copy-membership](#)

N4 | EDIT | 4. Lay out content and tab bar

Action: Replace the single entries.form layout assumption with one content layout per page plus any shared chrome. Apply bounds to the corresponding widget IDs and update group content size/clipping for each root. Set next.page_bar to a positive rectangle inside the client area, reserve space so it does not overlap content, and recompute both on resize. Zero default page_bar means no visible tab controls.

Why here: Declaring Page entries alone does not allocate visible tab-bar space or place their contents.

Source: [gui/shared/application.cpp:333](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [gui/shared/application.cpp:341](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:135](#) | [symbol](#) | [Read in PDF \(p. 54\)](#)

Read-only supplier evidence: [@gui-boundary/examples/application.hpp:156](#) | [symbol](#) | [Read in PDF \(p. 35\)](#)

Parameters and filled examples: [gui::Rect](#): left, top, width, and height

[Up to chart](#) | [Explore this step](#) | [page-layout](#)

N5 | EDIT | 5. Handle a page switch

Action: After the existing normalize_event gate, add a std::get_if<gui::PageEvent> branch and set view_.active_page to page->id. Preserve the final publish() call so selection and visibility reach the adapter. Use input-epoch invalidation if the switch changes authoritative input meaning; do not bypass event normalization.

Why here: The current handler has no PageEvent branch. Normalization already rejects unavailable pages, the active page, and page switches while a modal root is active.

Source: [gui/shared/application.cpp:85](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:163](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:159](#) | [symbol](#) | [Read in PDF \(p. 52\)](#)

Read-only supplier evidence: [@gui-boundary/examples/application.hpp:118](#) | [symbol](#) | [Read in PDF \(p. 60\)](#)

[Up to chart](#) | [Explore this step](#) | [page-switch](#)

N6 | REUSE | 6. Reuse tab rendering

Action: Publish the shared Snapshot. Existing supported adapters draw the tab bar from pages/page_bar and emit PageEvent; page membership determines content availability. Do not add a product-specific tab to each renderer. This recipe stays in existing shared source files, so it does not require a new compiler source entry.

Why here: Pages are an existing GUI contract capability. Supplier excerpts are evidence and examples, not edit targets.

Source: [gui/shared/application.cpp:357](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/presentation.hpp:35](#) | [symbol](#) | [Read in PDF \(p. 65\)](#)

[Up to chart](#) | [Explore this step](#) | [reuse-tabs](#)

N7 | REUSE | 7. Reuse adapter focus handling

Action: Keep active_page and all page memberships valid if pages are removed. Publishing the new snapshot lets the adapter clear focus that is no longer available; focus is not a Snapshot field. If a particular control should gain keyboard focus after a page switch, call adapter_.focus(...) after publish() succeeds, using a visible, enabled, focusable target in the newly presented page. Keyboard Tab follows available controls in snapshot order; it differs from the page tabs described here.

Why here: The existing adapter owns focus and availability. Add application focus policy only when the feature needs it.

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:403](#) | [symbol](#) | [Read in PDF \(p. 67\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:404](#) | [symbol](#) | [Read in PDF \(p. 67\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:247](#) | [symbol](#) | [Read in PDF \(p. 43\)](#)

[Up to chart](#) | [Explore this step](#) | [page-focus](#)

N8 | JUMP | Then add the page widgets

Action: Use Place a widget to set order, height, parent, and page. Use the widget/event/helper map for click behavior. For each later tab, add its page entry and group/content declarations; reuse the shared switching and layout policy introduced above unless its arrangement needs a new layout.

Why here: The tab path establishes navigation; each control still has a stable ID and its own behavior.

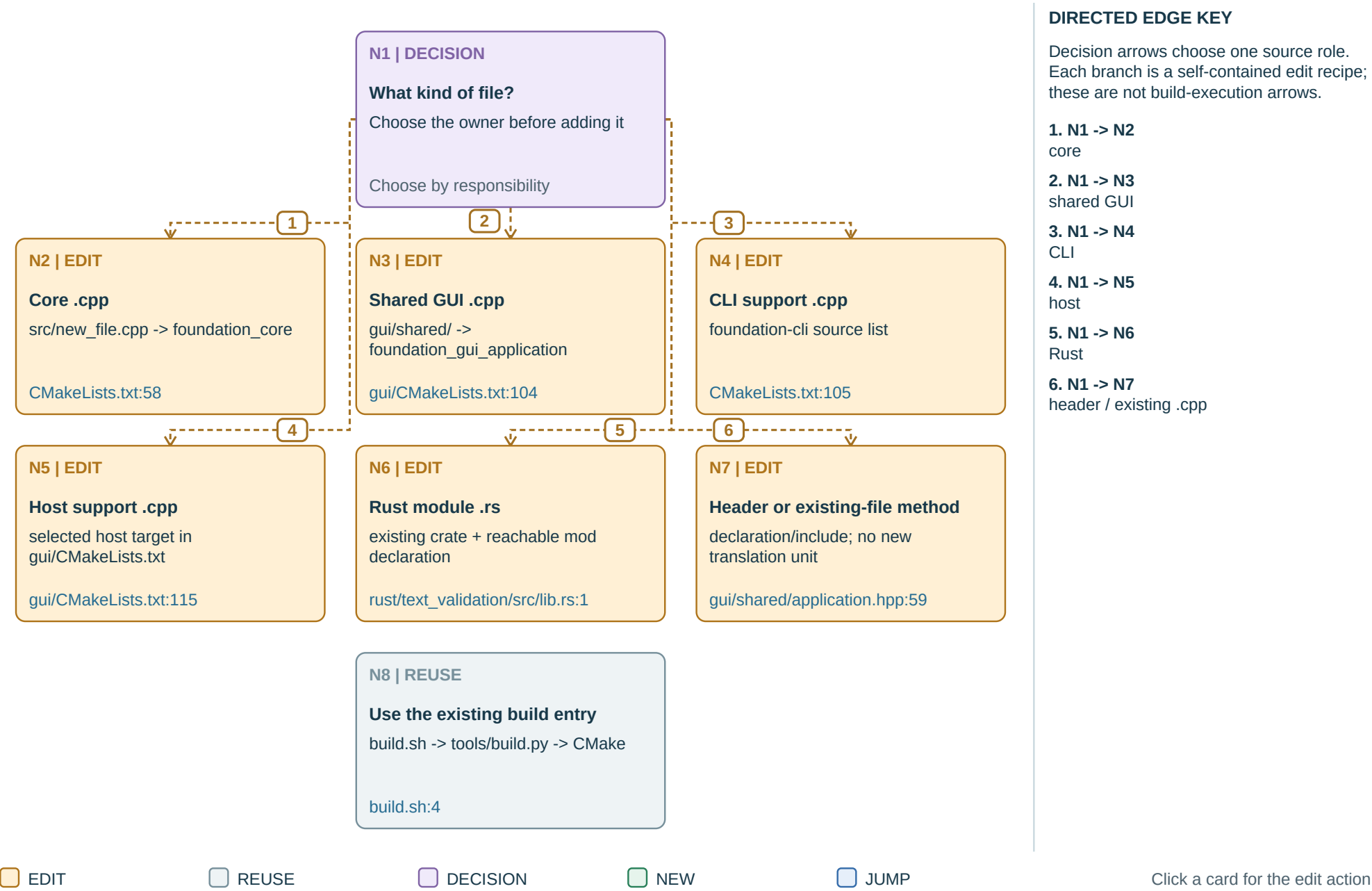
Source: [gui/shared/view_definition.hpp:9](#) | [symbol](#) | [Read in PDF \(p. 4\)](#)

Source: [gui/shared/application.cpp:101](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

[Drill down to the related edit map](#)

[Up to chart](#) | [Explore this step](#) | [tab-widget](#)

Where do I put a new file, and which compiler source list must I edit?



Actions behind this path

Choose the role of the file. Each branch names the file location and the exact source-registration edit. An existing function body or header declaration is different from a new translation unit.

[Up to chart](#) | The path is an editing plan. Runtime arrows are identified in the edge key.

N1 | DECISION | What kind of file?

Action: Decide whether the new code is core behavior, shared GUI behavior, CLI composition, host support, a Rust module, or a header. Then follow the corresponding branch.

Why here: The owning target determines where the compiler gets its source list.

[Up to chart](#) | [Explore this step](#) | choose-role

N2 | EDIT | Core .cpp

Action: Create the .cpp under src/. Add its source-relative path to `add_library(foundation_core STATIC ...)`, or an explicit `target_sources(foundation_core PRIVATE ...)` in the root CMakeLists.txt. Expose public declarations under `include/foundation/`.

Why here: A new file in src/ is not automatically compiled.

Source: CMakeLists.txt:58 | [Read in PDF](#) (p. 4)

Source: include/foundation/store.hpp:22 | [symbol](#) | [Read in PDF](#) (p. 18)

Parameters and filled examples: `target_sources`: target, visibility, then source paths

[Up to chart](#) | [Explore this step](#) | core-cpp

N3 | EDIT | Shared GUI .cpp

Action: Create the file under gui/shared/. In gui/CMakeLists.txt add `shared/new_file.cpp` to `foundation_gui_application`, alongside `shared/application.cpp`. Include the appropriate declaration from the caller. Preserve the existing platform selection for task executors.

Why here: The recursive GUI source GLOB watches boundary checks; it is not the compiler source list.

Source: gui/CMakeLists.txt:104 | [Read in PDF](#) (p. 7)

Source: gui/CMakeLists.txt:87

Parameters and filled examples: `target_sources`: target, visibility, then source paths

[Up to chart](#) | [Explore this step](#) | gui-cpp

N4 | EDIT | CLI support .cpp

Action: Add the CLI-specific implementation file to `add_executable(foundation-cli ...)` in root CMakeLists.txt, or `target_sources(foundation-cli PRIVATE ...)`. Put reusable operations in the core instead.

Why here: The CLI target composes the application and links `foundation::core`.

Source: CMakeLists.txt:105 | [Read in PDF](#) (p. 4)

Parameters and filled examples: `target_sources`: target, visibility, then source paths

[Up to chart](#) | [Explore this step](#) | cli-cpp

N5 | EDIT | Host support .cpp

Action: For an extra file in an existing host, use `target_sources(the-existing-host PRIVATE ...)` after its `foundation_gui_executable` declaration. For a new executable composition root, use the existing helper. Keep product widget IDs and command meaning on the shared Application side.

Why here: The helper accepts a target and one primary source; additional translation units belong to the actual target.

Source: gui/CMakeLists.txt:115 | [symbol](#)

Parameters and filled examples: `target_sources`: target, visibility, then source paths

[Up to chart](#) | [Explore this step](#) | host-cpp

N6 | EDIT | Rust module .rs

Action: Add the .rs file under `rust/text_validation/src/` and declare `mod new_module`; from `lib.rs` or another reachable module. Call or expose its functions as needed. The existing CMake input inventory already watches .rs files, but it does not make an undeclared Rust module reachable.

Why here: A new crate, dependency, or build.rs is a separate build-policy change, not this internal-module recipe.

Source: rust/text_validation/src/lib.rs:1 | [Read in PDF](#) (p. 120)

Source: cmake/RustComponent.cmake:143 | [symbol](#)

Source: tools/rust_build.py:335 | [symbol](#) | [Read in PDF](#) (p. 69)

[Up to chart](#) | [Explore this step](#) | rust-module

N7 | EDIT | Header or existing-file method

Action: Add the declaration to the owning header and include it from reachable code. If the implementation stays in an existing .cpp, there is no new source file to register. Inline/template definitions belong in a reachable header. Public include/foundation headers are covered by the existing include-directory install rule.

Why here: CMake source registration is required for a new .cpp, not merely for a new function declaration.

Source: [gui/shared/application.hpp:59](#) | [symbol](#) | [Read in PDF \(p. 14\)](#)

Source: [CMakeLists.txt:289](#)

[Up to chart](#) | [Explore this step](#) | header-only

N8 | REUSE | Use the existing build entry

Action: After an actual implementation change, use the project's existing manual build workflow and selected configuration. This documentation generator only shows that workflow and does not invoke it.

Why here: Registering an ordinary source in its target does not require adding a documentation build hook or changing CI, compiler installation, presets, or runners.

Source: [build.sh:4](#)

[Up to chart](#) | [Explore this step](#) | normal-build

N1 | DECISION

1. Identify the rule owner
Domain rule or private text format?

src/store.cpp:17

N2 | EDIT

2. State the accepted input
store.hpp | public behavior

include/foundation/store.hpp:28

N3 | EDIT

3. Match both text providers
If the private format changes

rust/text_validation/src/lib.rs:11

N4 | EDIT

4. Apply rules before mutation
Store::validate | messages

src/store.cpp:17

N5 | EDIT

5. Follow mirrored limits
Only if length/capacity changes

include/foundation/store.hpp:24

N6 | REUSE

6. Follow existing consumers
Core calls -> shared error status

gui/shared/application.cpp:49

DIRECTED EDGE KEY

Arrows show edit dependencies. Provider and limit steps are conditional, not runtime execution.

- 1. N1 -> N2
public behavior
- 2. N2 -> N3
if private format changes
- 3. N3 -> N4
shared statuses
- 4. N1 -> N4
domain rule
- 5. N4 -> N5
if bounds change
- 6. N5 -> N6
same accepted input
- 7. N4 -> N6
existing bounds unchanged

Click a card for the edit action

Actions behind this path

Start with the rule owner. A domain rule belongs in Store; a changed private text format must remain consistent across Rust and C++ providers. Input limits are mirrored in several places. Follow conditional edits only when the rule changes that contract; preserve validation before mutation.

[Up to chart](#) | The path is an editing plan. Runtime arrows are identified in the edge key.

N1 | DECISION | 1. Identify the rule owner

Action: For collection/domain meaning, edit Store validation or the owning core operation. For byte admissibility or the private ABI length limit, follow the provider-contract steps as well.

Why here: CLI, GUI and import should receive the same rule through core; a UI-only filter would leave other callers different.

Source: [src/store.cpp:17](#) | [symbol](#) | [Read in PDF \(p. 72\)](#)

Source: [include/foundation/store.hpp:28](#) | [symbol](#) | [Read in PDF \(p. 18\)](#)

[Up to chart](#) | [Explore this step](#) | rule-scope

N2 | EDIT | 2. State the accepted input

Action: Update the public input/failure contract if it changes. Preserve stable IDs, caller-serialized access and the guarantee that a failed operation leaves stored state unchanged.

Why here: A newcomer needs to know both what is accepted and what happens on rejection.

Source: [include/foundation/store.hpp:28](#) | [symbol](#) | [Read in PDF \(p. 18\)](#)

[Up to chart](#) | [Explore this step](#) | rule-contract

N3 | EDIT | 3. Match both text providers

Action: Change the Rust validator and explicit C++ compatibility validator together. If adding a numeric status, update the private C ABI declaration, Rust constants and C++ translation consistently.

Why here: Fresh configurations use Rust; selecting C++ must retain equivalent accepted input and status meaning.

Source: [rust/text_validation/src/lib.rs:11](#) | [symbol](#) | [Read in PDF \(p. 74\)](#)

Source: [src/text_validation.cpp:14](#) | [symbol](#) | [Read in PDF \(p. 76\)](#)

Source: [src/text_validation.h:17](#) | [Read in PDF \(p. 77\)](#)

[Up to chart](#) | [Explore this step](#) | rule-providers

N4 | EDIT | 4. Apply rules before mutation

Action: Implement the domain rule or translate provider statuses into an appropriate public exception. Keep validation before insertion/replacement, and update error text when its meaning changes.

Why here: Returning an old ASCII-only message for a different rule would mislead the caller.

Source: [src/store.cpp:17](#) | [symbol](#) | [Read in PDF \(p. 72\)](#)

Source: [src/store.cpp:33](#) | [symbol](#) | [Read in PDF \(p. 23\)](#)

Source: [src/store.cpp:47](#) | [symbol](#) | [Read in PDF \(p. 78\)](#)

[Up to chart](#) | [Explore this step](#) | rule-core

N5 | EDIT | 5. Follow mirrored limits

Action: Review the public constants, private ABI/provider bounds, editor text policy and captured-task bounds. Do not increase a public constant alone; preserve agreement among the relevant contracts.

Why here: The current private text ABI is bounded to 256 bytes, and tasks separately enforce 256-byte records.

Source: [include/foundation/store.hpp:24](#) | [symbol](#) | [Read in PDF \(p. 18\)](#)

Source: [gui/shared/application.cpp:18](#) | [symbol](#) | [Read in PDF \(p. 20\)](#)

Source: [gui/shared/task.hpp:30](#) | [symbol](#) | [Read in PDF \(p. 81\)](#)

[Up to chart](#) | [Explore this step](#) | rule-bounds

N6 | REUSE | 6. Follow existing consumers

Action: Reuse Store calls from CLI, GUI and import. Shared GUI code already catches invalid input and projects its error status; inspect those paths to see the user-facing result of the changed rule.

Why here: The validation remains one reusable behavior rather than a separate rule per frontend.

Source: [gui/shared/application.cpp:49](#) | [symbol](#) | [Read in PDF \(p. 11\)](#)

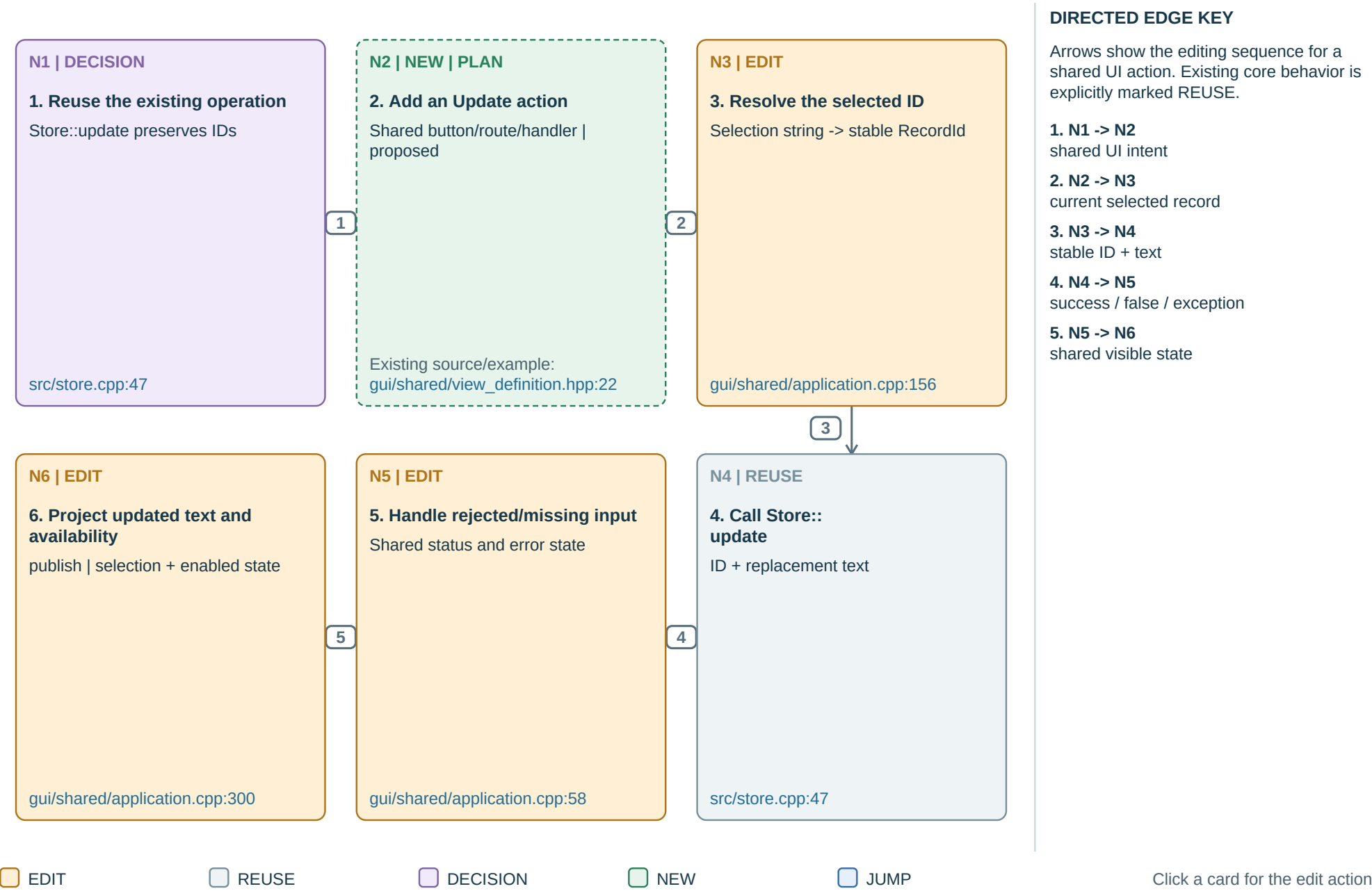
Source: [gui/shared/application.cpp:58](#) | [symbol](#) | [Read in PDF \(p. 11\)](#)

Source: [gui/shared/application.cpp:300](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [src/main.cpp:36](#) | [symbol](#) | [Read in PDF \(p. 83\)](#)

[Up to chart](#) | [Explore this step](#) | rule-consumers

How do I replace the selected record text while preserving its identity?



Actions behind this path

Store::update already provides this core operation. Add shared UI intent and selection resolution, then reuse it. get/snapshot return copies; editing a copy does not change Store. Erase plus add would allocate a different ID. The proposed edit action and handler below do not already exist.

[Up to chart](#) | The path is an editing plan. Runtime arrows are identified in the edge key.

N1 | DECISION | 1. Reuse the existing operation

Action: Use Store::update for replacement text on an existing record. Add a different core API only if the required domain behavior cannot be expressed by the current contract.

Why here: The same API can serve a future CLI command; the current CLI has no edit command or persistent store.

Source: [src/store.cpp:47](#) | [symbol](#) | [Read in PDF \(p. 78\)](#)

Source: [include/foundation/store.hpp:32](#) | [symbol](#) | [Read in PDF \(p. 18\)](#)

Source: [src/main.cpp:13](#) | [symbol](#) | [Read in PDF \(p. 83\)](#)

[Up to chart](#) | [Explore this step](#) | [edit-owner](#)

N2 | NEW | 2. Add an Update action

Planned addition to this application. It is not implemented yet; links show existing source and examples.

Action: Declare a supported button such as entries.update, add its target-ID Activate case, and declare a proposed Application::edit_selected() member implemented in application.cpp. Reuse the current widget/edit pattern.

Why here: Widget identity and command meaning belong to shared Application. Existing application.cpp is already compiled.

Existing source/example: [gui/shared/view_definition.hpp:22](#) | [Read in PDF \(p. 119\)](#)

Source: [gui/shared/application.cpp:101](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.hpp:59](#) | [symbol](#) | [Read in PDF \(p. 14\)](#)

Source: [gui/CMakeLists.txt:104](#) | [Read in PDF \(p. 7\)](#)

[Drill down to the related edit map](#)

[Up to chart](#) | [Explore this step](#) | [edit-intent](#)

N3 | EDIT | 3. Resolve the selected ID

Action: Read entries.list selected state and entries.editor text. If no selection exists, do not mutate. Match the selected string against std::to_string(record.id) from authoritative Store records, following the existing removal pattern, and retain the matching RecordId. Call advance_input_epoch() before accepted semantic mutation; keep that increment even if later publication fails.

Why here: A selected string is an identity, not a vector index. Returned snapshots are owning copies.

Source: [gui/shared/application.cpp:156](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:119](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:67](#) | [symbol](#) | [Read in PDF \(p. 87\)](#)

Source: [gui/shared/application.cpp:51](#) | [symbol](#) | [Read in PDF \(p. 11\)](#)

[Up to chart](#) | [Explore this step](#) | [edit-selection](#)

N4 | REUSE | 4. Call Store::update

Action: Call entries_.update(resolved_id, replacement_text) from the new shared handler. Keep the existing core implementation when its behavior fits the requirement.

Why here: update validates text first, returns false for a missing ID when the text is valid, and prepares a replacement string before swapping it into place. The record ID stays unchanged.

Source: [src/store.cpp:47](#) | [symbol](#) | [Read in PDF \(p. 78\)](#)

Source: [include/foundation/store.hpp:32](#) | [symbol](#) | [Read in PDF \(p. 18\)](#)

Parameters and filled examples: Store::update: stable record ID and replacement text

[Up to chart](#) | [Explore this step](#) | [edit-update](#)

N5 | EDIT | 5. Handle rejected/missing input

Action: Handle invalid_argument as user input rejection and a false return as a missing record. Set a useful shared status; decide whether successful editing clears or retains editor text. Do not report a false return as success.

Why here: Invalid replacement text can throw before missing-ID lookup, so both outcomes need explicit handling.

Source: [gui/shared/application.cpp:58](#) | [symbol](#) | [Read in PDF \(p. 11\)](#)

Source: [gui/shared/application.cpp:49](#) | [symbol](#) | [Read in PDF \(p. 11\)](#)

Source: [src/store.cpp:47](#) | [symbol](#) | [Read in PDF \(p. 78\)](#)

[Up to chart](#) | [Explore this step](#) | [edit-result](#)

N6 | EDIT | 6. Project updated text and availability

Action: Add Update enabled-state logic based on valid selection and the intended edit policy. Reuse publish to rebuild rows from Store and retain/clear selection consistently. The existing handle publishes after dispatch returns.

Why here: UI text becomes visible through the shared snapshot; core mutation alone does not define action availability.

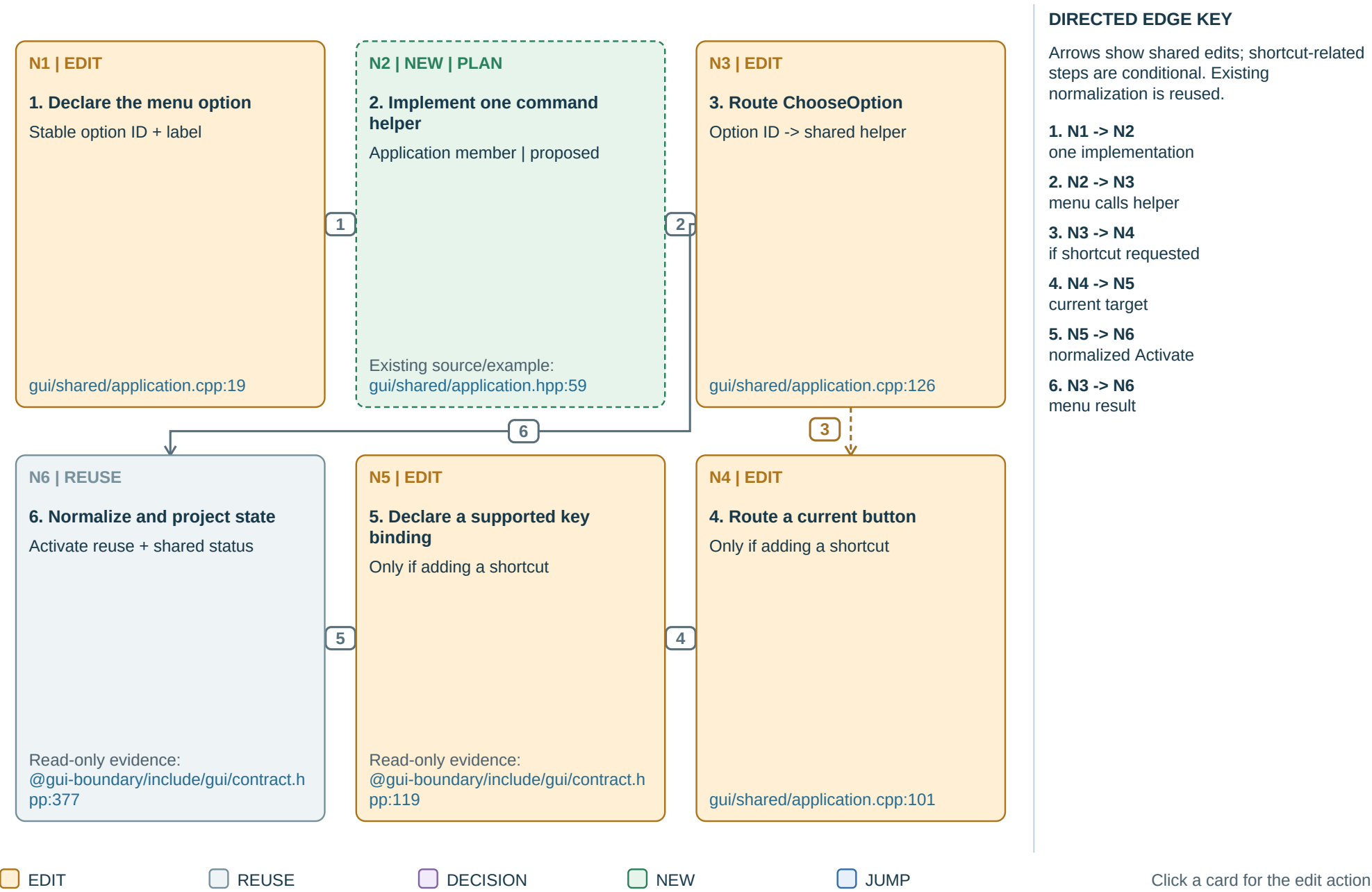
Source: [gui/shared/application.cpp:300](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [gui/shared/application.cpp:352](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [gui/shared/application.cpp:163](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

[Up to chart](#) | [Explore this step](#) | [edit-view](#)

How do I add an action to the menu and reuse its behavior from a shortcut?



Actions behind this path

Add shared command intent and route it to one helper. Menu options arrive as ChooseOption; shortcuts are normalized into Activate on a current button. The pinned shortcut keys are Escape, Enter and F1-F12 with modifier flags. A proposed command/helper does not already exist.

[Up to chart](#) | The path is an editing plan. Runtime arrows are identified in the edge key.

N1 | EDIT | 1. Declare the menu option

Action: Append an option to entries.options in the Application constructor, using a distinct command ID and the desired label. Keep availability derived from shared state when it changes over time.

Why here: Options declare user intent; generic renderers do not implement product command meaning.

Source: [gui/shared/application.cpp:19](#) | [symbol](#) | [Read in PDF \(p. 20\)](#)

[Up to chart](#) | [Explore this step](#) | [command-option](#)

N2 | NEW | 2. Implement one command helper

Planned addition to this application. It is not implemented yet; links show existing source and examples.

Action: Declare and implement a proposed shared helper for the command. Put reusable record/domain behavior in core and call it from the helper; keep status and menu meaning in Application. Call advance_input_epoch() before accepted semantic mutation, preserving the increment if publication fails.

Why here: A menu action and a button/shortcut can call the same helper.

Existing source/example: [gui/shared/application.hpp:59](#) | [Read in PDF \(p. 14\)](#)

Source: [gui/shared/application.cpp:49](#) | [symbol](#) | [Read in PDF \(p. 11\)](#)

Source: [gui/shared/application.cpp:67](#) | [symbol](#) | [Read in PDF \(p. 87\)](#)

[Drill down to the related edit map](#)

[Up to chart](#) | [Explore this step](#) | [command-helper](#)

N3 | EDIT | 3. Route ChooseOption

Action: Add an input.id case in the ChooseOption branch and call the helper. For another menu, use unambiguous command IDs or explicitly check widget->target.id before interpreting its options.

Why here: The current menu dispatch checks option IDs alone; repeated IDs in different menus could otherwise share unintended meaning.

Source: [gui/shared/application.cpp:126](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:127](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

[Up to chart](#) | [Explore this step](#) | [command-route](#)

N4 | EDIT | 4. Route a current button

Action: Provide a current button for the action, and route its target-ID Activate case to the same helper. Preserve enabled/visible state so shortcut eligibility matches the action.

Why here: The retained contract requires every shortcut to target a current button, not a menu option ID.

Source: [gui/shared/application.cpp:101](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/view_definition.hpp:22](#) | [Read in PDF \(p. 119\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:321](#) | [symbol](#) | [Read in PDF \(p. 43\)](#)

[Up to chart](#) | [Explore this step](#) | [command-button](#)

N5 | EDIT | 5. Declare a supported key binding

Action: Initialize view_.key_bindings with a supported ShortcutKey, the current button WidgetKey and control/shift/alt flags. Use a unique key/modifier combination; arbitrary letter keys are not in this pinned vocabulary.

Why here: A shortcut is declared in the shared Snapshot, independent of a native toolkit callback.

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:119](#) | [symbol](#) | [Read in PDF \(p. 89\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:118](#) | [Read in PDF \(p. 90\)](#)

Parameters and filled examples: [gui::KeyBinding](#): key, button target, and modifiers

[Up to chart](#) | [Explore this step](#) | [command-binding](#)

N6 | REUSE | 6. Normalize and project state

Action: Reuse `normalize_event`: it maps a matched `ShortcutEvent` to the button `Activate` path. Keep normalization before command dispatch, and update `publish` if the command changes status or availability.

Why here: Menu, button and shortcut behavior can converge on the same shared action, with normal stale/disabled input rejection.

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:377](#) | [symbol](#) | [Read in PDF](#) (p. 91)

Source: [gui/shared/application.cpp:85](#) | [symbol](#) | [Read in PDF](#) (p. 5)

Source: [gui/shared/application.cpp:300](#) | [symbol](#) | [Read in PDF](#) (p. 26)

[Up to chart](#) | [Explore this step](#) | [command-project](#)

Where do I change the work and its displayed result while retaining cancellation and responsiveness?



DIRECTED EDGE KEY

Arrows show edit dependencies across algorithm, input, completion and shared lifecycle; these are not thread-execution arrows.

1. **N1 -> N2**
meaning + work units
2. **N2 -> N3**
owned inputs
3. **N3 -> N4**
executor interface
4. **N4 -> N5**
value-only progress
5. **N5 -> N6**
visible result + close

EDIT
REUSE
DECISION
NEW
JUMP

Click a card for the edit action

Actions behind this path

The current task counts non-space bytes from captured record text. Computation is owned/value-only; native and cooperative schedulers reuse it. If result semantics change, review completion guards that currently require `result <= processed` and `complete == (processed == total)`.

[Up to chart](#) | The path is an editing plan. Runtime arrows are identified in the edge key.

N1 | DECISION | 1. Define the result/progress contract

Action: Keep the current processed/total/result counters if the new work has the same meaning. If changing the value shape or invariants, update `TaskUpdate` and every consumer consistently.

Why here: The current completion validator encodes counting semantics; a different numeric result may legitimately exceed processed.

Source: [gui/shared/task.hpp:12](#) | [symbol](#) | [Read in PDF \(p. 93\)](#)

Source: [gui/shared/application.cpp:281](#) | [symbol](#) | [Read in PDF \(p. 94\)](#)

[Up to chart](#) | [Explore this step](#) | task-result

N2 | EDIT | 2. Change bounded computation

Action: Modify the work in `TextTask` while preserving owned input, bounded work per advance call and cancellation/shutdown behavior. Update totals when the work unit changes.

Why here: The existing result increments for non-space bytes; changing only the UI label does not change the computation.

Source: [gui/shared/task.hpp:39](#) | [symbol](#) | [Read in PDF \(p. 96\)](#)

Source: [gui/shared/task.hpp:45](#) | [symbol](#) | [Read in PDF \(p. 96\)](#)

[Up to chart](#) | [Explore this step](#) | task-work

N3 | EDIT | 3. Capture inputs and route start/cancel

Action: Adapt the shared Start branch if input capture changes. Copy authoritative record text into owned task input, start through `TaskExecutor`, and keep cancellation through the existing Cancel branch.

Why here: Later edits to live records must not mutate already captured work, and workers must not retain `Application` or adapter references.

Source: [gui/shared/application.cpp:102](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:107](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:113](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

[Up to chart](#) | [Explore this step](#) | task-start

N4 | REUSE | 4. Reuse scheduling and tick delivery

Action: Retain the value-oriented `TaskExecutor` boundary for ordinary algorithm changes. Existing ticks advance/poll work and deliver updates on the shared application side.

Why here: Scheduling is selected once for the platform; task logic does not need a separate implementation in each GUI renderer.

Source: [gui/shared/task.hpp:58](#) | [symbol](#) | [Read in PDF \(p. 99\)](#)

Source: [gui/shared/application.cpp:277](#) | [symbol](#) | [Read in PDF \(p. 100\)](#)

[Up to chart](#) | [Explore this step](#) | task-schedule

N5 | EDIT | 5. Interpret completion honestly

Action: Update result/progress text in `complete_task`. Review changed result invariants while retaining generation matching, stale/cancelled completion rejection and monotonic progress where required.

Why here: Current guards reject `result > processed`, and require `complete` to mean `processed == total`; keep or deliberately revise these with the contract.

Source: [gui/shared/application.cpp:281](#) | [symbol](#) | [Read in PDF \(p. 94\)](#)

Source: [gui/shared/application.cpp:285](#) | [symbol](#) | [Read in PDF \(p. 94\)](#)

[Up to chart](#) | [Explore this step](#) | task-completion

N6 | REUSE | 6. Project controls and close owned work

Action: Reuse shared Start/Cancel enabled states and progress projection; edit them if the new task policy changes. Preserve shutdown of the executor before application state destruction.

Why here: Responsiveness includes cancellation, late update rejection and complete owned-work cleanup, not just showing progress.

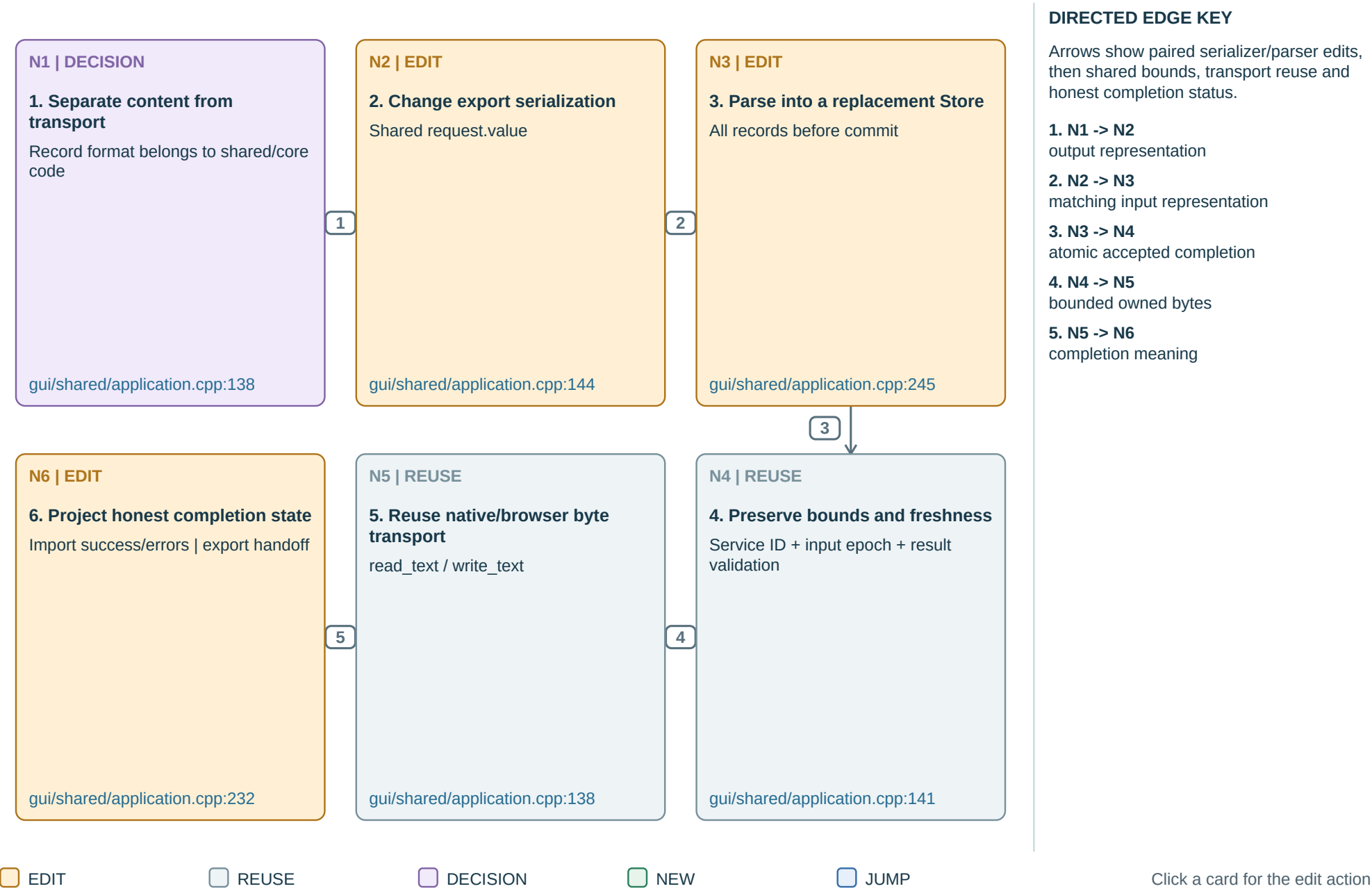
Source: [gui/shared/application.cpp:353](#) | [symbol](#) | [Read in PDF](#) (p. 26)

Source: [gui/shared/application.cpp:354](#) | [symbol](#) | [Read in PDF](#) (p. 26)

Source: [gui/shared/application.cpp:271](#) | [symbol](#) | [Read in PDF](#) (p. 102)

[Up to chart](#) | [Explore this step](#) | [task-lifecycle](#)

Where do I change record serialization without duplicating file transport across hosts?



Actions behind this path

The current format is one text record per line, with CR stripped on import. Shared code owns format and record meaning; generic hosts transfer bytes. Import stages a replacement Store before commit. Export reports handoff to the host, which is not a promise of durable storage.

[Up to chart](#) | The path is an editing plan. Runtime arrows are identified in the edge key.

N1 | DECISION | 1. Separate content from transport

Action: Define the desired record representation and malformed-input policy. Keep format interpretation in shared/core code; leave paths, browser file selection and byte movement in generic services.

Why here: Changing a content format does not require copying application meaning into every backend.

Source: [gui/shared/application.cpp:138](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:232](#) | [symbol](#) | [Read in PDF \(p. 104\)](#)

Source: [include/foundation/store.hpp:28](#) | [symbol](#) | [Read in PDF \(p. 18\)](#)

[Up to chart](#) | [Explore this step](#) | format-owner

N2 | EDIT | 2. Change export serialization

Action: Replace the current record-text-plus-newline serialization in the export command branch. Check the complete serialized byte count against the service request bound before enqueueing.

Why here: The serialized representation, not merely the number of records, determines transfer size.

Source: [gui/shared/application.cpp:144](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:145](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

[Up to chart](#) | [Explore this step](#) | format-export

N3 | EDIT | 3. Parse into a replacement Store

Action: Replace line parsing in `complete_service` with the matching parser. Populate a temporary copy through core record operations, then commit only after every record succeeds. Preserve monotonic IDs and clear obsolete selection on successful replacement.

Why here: Mutating the live Store while parsing would expose a partial import on rejection.

Source: [gui/shared/application.cpp:245](#) | [symbol](#) | [Read in PDF \(p. 104\)](#)

Source: [gui/shared/application.cpp:249](#) | [symbol](#) | [Read in PDF \(p. 104\)](#)

Source: [gui/shared/application.cpp:257](#) | [symbol](#) | [Read in PDF \(p. 104\)](#)

[Up to chart](#) | [Explore this step](#) | format-import

N4 | REUSE | 4. Preserve bounds and freshness

Action: Retain request limits, pending-service identity checks and shutdown/cancellation/error paths. Keep authoritative input-epoch invalidation around request/completion changes so stale inputs cannot apply against changed meaning. A deliberately different size bound is a contract change.

Why here: A format parser alone does not authorize an unmatched service result or stale input.

Source: [gui/shared/application.cpp:141](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/shared/application.cpp:235](#) | [symbol](#) | [Read in PDF \(p. 104\)](#)

Source: [gui/shared/application.cpp:67](#) | [symbol](#) | [Read in PDF \(p. 87\)](#)

Source: [gui/shared/application.cpp:237](#) | [symbol](#) | [Read in PDF \(p. 104\)](#)

[Up to chart](#) | [Explore this step](#) | format-freshness

N5 | REUSE | 5. Reuse native/browser byte transport

Action: Keep existing `read_text/write_text` capabilities for valid UTF-8 text formats within the 64 KiB transfer ceiling. Arbitrary binary data requires a different capability or text encoding within that bound. Inspect generic transfer behavior; do not move record parsing into the host.

Why here: Native and browser hosts own their file-selection/transfer authority while the application owns content meaning.

Source: [gui/shared/application.cpp:138](#) | [symbol](#) | [Read in PDF \(p. 5\)](#)

Source: [gui/host/file_services.hpp:66](#) | [symbol](#) | [Read in PDF \(p. 109\)](#)

Source: [gui/host/file_services.mjs:29](#) | [symbol](#) | [Read in PDF \(p. 114\)](#)

Source: [gui/host/file_services.hpp:70](#) | [symbol](#) | [Read in PDF \(p. 109\)](#)

[Up to chart](#) | [Explore this step](#) | format-transport

N6 | EDIT | 6. Project honest completion state

Action: Set shared import/parse/error status and project the committed rows.
Keep export wording honest: the existing completion says Export handed to host, not that data is durably saved. Preserve cancelled and transport-error outcomes.

Why here: The visible result must distinguish a successfully parsed replacement, rejected content and host handoff.

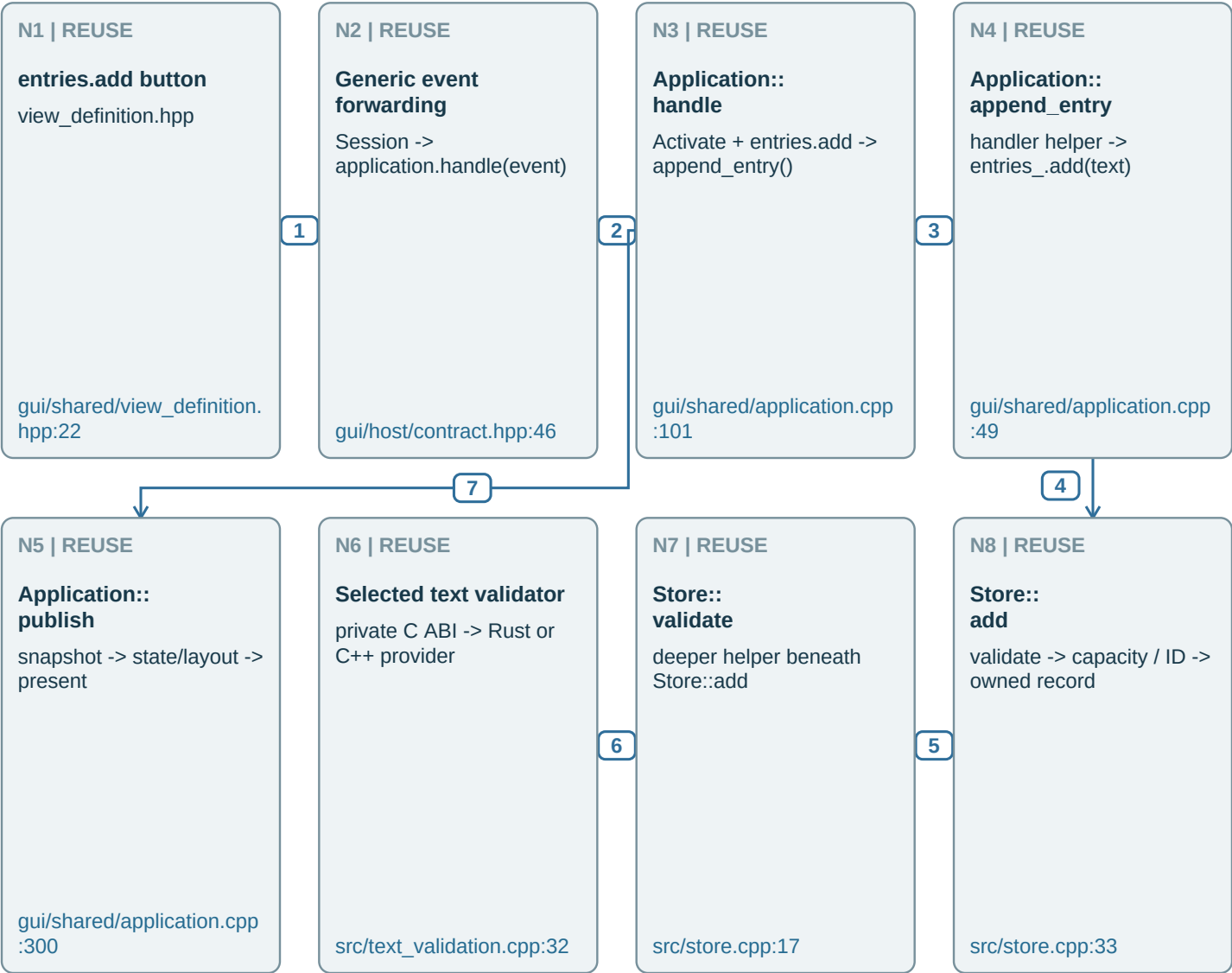
Source: [gui/shared/application.cpp:232](#) | [symbol](#) | [Read in PDF \(p. 104\)](#)

Source: [gui/shared/application.cpp:300](#) | [symbol](#) | [Read in PDF \(p. 26\)](#)

Source: [gui/shared/application.cpp:242](#) | [symbol](#) | [Read in PDF \(p. 104\)](#)

[Up to chart](#) | [Explore this step](#) | [format-status](#)

Which existing functions does an Add-entry click reach? These are existing functions; choose an edit point by the behavior you want to change.



DIRECTED EDGE KEY

Arrows show named calls or explicit event/return ordering in this example, not a complete runtime graph. The adapter event bridge is a contract relationship. Provider selection remains conditional.

- 1. N1 -> N2
adapter emits Activate
- 2. N2 -> N3
forwards event
- 3. N3 -> N4
1. calls handler helper
- 4. N4 -> N8
entries_.add(text)
- 5. N8 -> N7
validate(text)
- 6. N7 -> N6
detail::validate_text
- 7. N3 -> N5
2. after normal dispatch

Actions behind this path

This is the source-backed example behind the edit recipe. Follow the call chain into the deeper helper; `handle()` publishes after its event dispatch returns. It also routes `SubmitText` to `append_entry()`.

[Up to chart](#) | The path is an editing plan. Runtime arrows are identified in the edge key.

N1 | REUSE | `entries.add` button

Action: The row declares the button ID and label. A supported adapter turns activation into a `WidgetEvent` targeting that ID with `gui::Activate` input.

Why here: Use the same ID in declaration and dispatch.

Source: [gui/shared/view_definition.hpp:22](#) | [Read in PDF](#) (p. 119)

[Up to chart](#) | [Explore this step](#) | `add-widget`

N2 | REUSE | Generic event forwarding

Action: The native Session adapter callback forwards events to the application. Browser hosts have their own transport bridge but reach the same shared application event logic.

Why here: A feature click does not need separate business handlers for each backend.

Source: [gui/host/contract.hpp:46](#) | [symbol](#) | [Read in PDF](#) (p. 22)

[Up to chart](#) | [Explore this step](#) | `event-bridge`

N3 | REUSE | `Application::handle`

Action: After normalization, the `Activate` branch matches `widget->target.id`. The `entries.add` case calls `append_entry()`. After normal visitor completion, `handle` calls `publish()`.

Why here: This is the precise routing edit point for a new button ID.

Source: [gui/shared/application.cpp:101](#) | [symbol](#) | [Read in PDF](#) (p. 5)

Source: [gui/shared/application.cpp:163](#) | [symbol](#) | [Read in PDF](#) (p. 5)

[Up to chart](#) | [Explore this step](#) | `dispatch`

N4 | REUSE | `Application::append_entry`

Action: Read editor text, call the core operation, clear the editor on success, and set shared status. Caught `invalid_argument` and `length_error` exceptions update shared error status; other exceptions can escape. `append_entry` does not itself call `publish`.

Why here: This is the model for the function below your event handler.

Source: [gui/shared/application.cpp:49](#) | [symbol](#) | [Read in PDF](#) (p. 11)

[Up to chart](#) | [Explore this step](#) | `app-operation`

N5 | REUSE | `Application::publish`

Action: After normal dispatch completion, handle calls `publish`. It projects `Store` records into list rows and updates layout, status, and enabled flags before `adapter_.present(next)`.

Why here: Edit this function when new state needs a visible projection.

Source: [gui/shared/application.cpp:300](#) | [symbol](#) | [Read in PDF](#) (p. 26)

Source: [gui/shared/application.cpp:357](#) | [symbol](#) | [Read in PDF](#) (p. 26)

Source: [gui/shared/application.cpp:163](#) | [symbol](#) | [Read in PDF](#) (p. 5)

[Up to chart](#) | [Explore this step](#) | `publish`

N6 | REUSE | Selected text validator

Action: `detail::validate_text` calls `foundation_text_validate_v1`. The selected provider implements that private ABI; fresh configurations default to Rust. Both providers preserve the same text contract.

Why here: Follow these deeper links when changing validation itself.

Source: [src/text_validation.cpp:32](#) | [symbol](#) | [Read in PDF](#) (p. 116)

Source: [rust/text_validation/src/lib.rs:28](#) | [symbol](#) | [Read in PDF](#) (p. 117)

Source: [rust/text_validation/src/lib.rs:11](#) | [symbol](#) | [Read in PDF](#) (p. 74)

Source: [src/text_validation.cpp:14](#) | [symbol](#) | [Read in PDF](#) (p. 76)

[Up to chart](#) | [Explore this step](#) | `provider`

N7 | REUSE | Store::validate

Action: Store::add calls validate(text). This helper calls detail::validate_text and translates private status codes into the public C++ exception contract.

Why here: This is a concrete function called by the function beneath the event.

Source: [src/store.cpp:17](#) | [symbol](#) | [Read in PDF \(p. 72\)](#)

[Up to chart](#) | [Explore this step](#) | domain-helper

N8 | REUSE | Store::add

Action: Validate text before insertion, check capacity and identifier availability, then insert an owned record and advance the ID. Callers receive the new record ID.

Why here: Reusable domain rules are implemented here, outside GUI event routing.

Source: [src/store.cpp:33](#) | [symbol](#) | [Read in PDF \(p. 23\)](#)

[Up to chart](#) | [Explore this step](#) | core-operation

Parameter reference and filled examples

These shared guides explain the values used by the linked edit steps. Each guide appears once. Read its notation note before using the filled example; argument positions, named fields, defaults, and optional entries are distinguished there.

Select a guide below, then use its return links to continue at an edit step or its chart.

Parameter guide	Use it for
ViewDefinition: the widget row, field by field	Each top-level comma advances to the next struct field. The empty strings are different fields, not interchangeable padding.
gui::Rect: left, top, width, and height	For <code>page_bar</code> and widget bounds, <code>Rect</code> uses logical client coordinates. The last two fields are extents, not the right and bottom edges.
Widget membership: name the page and parent explicitly	These are named member assignments, not positional arguments. A widget ID, page ID, and parent ID have different jobs.
gui::Page: a tab ID, caption, and availability	A <code>Page</code> describes one tab. Page IDs are for membership and events; labels are for people reading the tab bar.
target_sources: target, visibility, then source paths	This CMake command appends files to an existing target. It uses whitespace-separated arguments, not C++ commas.
Store::update: stable record ID and replacement text	Replace text on an existing record while retaining its ID. The <code>bool</code> distinguishes successful replacement from a missing record when the replacement text is valid.
gui::KeyBinding: key, button target, and modifiers	A shared shortcut targets a current button <code>WidgetKey</code> . Normalization converts the shortcut to that button's <code>Activate</code> event.

ViewDefinition: the widget row, field by field

Each top-level comma advances to the next struct field. The empty strings are different fields, not interchangeable padding.

Return to edit steps: [widget / N1 \(chart\)](#) | [widget-placement / N2 \(chart\)](#) | [tabs / N2 \(chart\)](#)

Declaration and examples:

Source: [gui/shared/view_definition.hpp:9](#) | [symbol](#) | [Read in PDF \(p. 4\)](#)

Source: [gui/shared/view_definition.hpp:22](#) | [Read in PDF \(p. 119\)](#)

Synopsis

```
ViewDefinition{ id, kind, height, text, label, placeholder,  
               [bold, [wrap, [remove_extension]]] }
```

Notation: Reading notation, not C++ to paste: square brackets mean optional trailing fields. The filled example below is a C++ aggregate expression. The first six fields stay explicit in this recipe.

Numbered parameters

#	Parameter	Type	Meaning	Default	Filled example value
1	id	std::string_view	Stable widget identity; use the same ID when routing events.	No member initializer; specify explicitly in this recipe.	"entries.custom"
2	kind	gui::Kind	Which supported control to create. This example creates a button.	No member initializer; specify explicitly in this recipe.	gui::Kind::button
3	height	double	Fixed row height in shared logical units; 0 makes a zero-height leaf in this layout.	No member initializer; specify explicitly in this recipe.	32
4	text	std::string_view	Initial state.text, such as label content or editor contents.	No member initializer; specify explicitly in this recipe.	""
5	label	std::string_view	Initial state.label; the button caption in this example.	No member initializer; specify explicitly in this recipe.	"Run custom action"
6	placeholder	std::string_view	Hint for an empty editor or another control that supports it.	No member initializer; specify explicitly in this recipe.	""

#	Parameter	Type	Meaning	Default	Filled example value
7	bold	bool	Initial font weight.	false	false
8	wrap	gui::TextWrap	Text wrapping mode for text that uses shared measurement.	gui::TextWrap::none	gui::TextWrap::none
9	remove_extension	bool	Reserve for the existing optional Remove feature; ordinary controls use false.	false	false

Filled example: an ordinary button row (illustrative new ID)

```
ViewDefinition{
  "entries.custom",      // 1. id
  gui::Kind::button,     // 2. kind
  32,                    // 3. height
  "",                    // 4. text
  "Run custom action",   // 5. label
  "",                    // 6. placeholder
  false,                 // 7. bold
  gui::TextWrap::none,   // 8. wrap
  false                  // 9. remove_extension
}
```

- Insert this expression as an element of view_definition, with a separating comma as needed. It does not implement the click handler.
- The three trailing values may be omitted to use their declared defaults. C++ can also value-initialize earlier omitted fields, but an empty ID is not a valid widget identity.
- This is the current nine-field schema. It has no page or parent field; those are explicit proposed additions in the tab edit path.
- gui::Kind::button is a scoped enum value, not another parameter. The two colons qualify its name; commas separate aggregate fields.

gui::Rect: left, top, width, and height

For page_bar and widget bounds, Rect uses logical client coordinates. The last two fields are extents, not the right and bottom edges.

Return to edit steps: [widget-placement / N3 \(chart\)](#) | [widget-placement / N5 \(chart\)](#) | [tabs / N4 \(chart\)](#)

Declaration and examples:

Read-only supplier evidence: [@gui-boundary/include/gui/geometry.hpp:13](#) | [symbol](#) | [Read in PDF \(p. 34\)](#)

Read-only supplier evidence: [@gui-boundary/examples/application.hpp:156](#) | [symbol](#) | [Read in PDF \(p. 35\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:95](#) | [symbol](#) | [Read in PDF \(p. 39\)](#)

Synopsis

```
gui::Rect{ x, y, width, height }
```

Notation: The names indicate positional fields; replace them with values. This also explains an untyped brace list assigned to page_bar or bounds.

Numbered parameters

#	Parameter	Type	Meaning	Default	Filled example value
1	x	double	Left edge; for this page_bar example, measured from the client area's left edge.	0	16
2	y	double	Top edge; for this page_bar example, measured from the client area's top edge.	0	448
3	width	double	Horizontal extent; 608 means right edge is 16 + 608 = 624.	0	608
4	height	double	Vertical extent; 24 means bottom edge is 448 + 24 = 472.	0	24

Filled example: a tab bar for a 640 by 480 client area

```
next.page_bar = gui::Rect{
  16,    // 1. x: left edge
  448,   // 2. y: top edge (480 - 32)
  608,   // 3. width (not right edge)
  24     // 4. height (not bottom edge)
};
```

- These fixed numbers explain the fields; real `publish()` layout must recompute bounds for the current client size and reserve space for page contents.
- This illustration has a 16-unit left/right inset and ends at `y=472`, leaving 8 units below in a 480-unit client area.
- These numbers evaluate the retained example: `y = max(0, 480 - 32)` and `width = max(0, 640 - 32)`. The two arguments to `max` are nested inside those individual `Rect` fields.
- Check the owning field's coordinate convention: `group content_clip` uses `x/y` relative to its group's top-left corner. It is not a client-space `page_bar` rectangle.
- Coordinates must be finite and within contract limits; extents must be nonnegative. The visible tab bar needs positive width and height. Rectangles are half-open.

Widget membership: name the page and parent explicitly

These are named member assignments, not positional arguments. A widget ID, page ID, and parent ID have different jobs.

Return to edit steps: [widget-placement / N4 \(chart\)](#) | [tabs / N2 \(chart\)](#) | [tabs / N3 \(chart\)](#)

Declaration and examples:

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:19](#) | [symbol](#) | [Read in PDF \(p. 51\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:82](#) | [symbol](#) | [Read in PDF \(p. 41\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:266](#) | [symbol](#) | [Read in PDF \(p. 43\)](#)

Synopsis

```
widget.spec.key.id = WIDGET_ID;
widget.spec.page = PAGE_ID;
widget.spec.parent = PARENT_WIDGET_ID;
```

Notation: The uppercase words are explanatory placeholders. The filled example uses actual string literals and separate statements.

Numbered parameters

#	Parameter	Type	Meaning	Default	Filled example value
1	spec.key.id	std::string	Unique widget ID used for lookup and event routing.	Empty string until assigned; supply a valid ID.	"settings.save"
2	spec.page	std::string	Declared page ID. An empty string deliberately means shared chrome.	Empty string	"settings"
3	spec.parent	std::string	ID of an earlier group on the same page. Empty means a root widget.	Empty string	"settings.form"

Filled example: a page-owned root group and its button

```
gui::Widget group;
group.spec.key.id = "settings.form";
group.spec.kind = gui::Kind::group;
group.spec.page = "settings";
group.spec.parent = ""; // root: no parent

gui::Widget button;
button.spec.key.id = "settings.save"; // widget ID
button.spec.kind = gui::Kind::button;
button.spec.page = "settings"; // page ID
button.spec.parent = "settings.form"; // parent ID
button.state.label = "Save settings";

view_.widgets.push_back(group); // parent first
view_.widgets.push_back(button); // child second
```

- Illustrative relationship only: declare the settings page first. The application recipe implements this mapping in `ViewDefinition/add()` instead of adding an unrelated second creation path.
- The current `ViewDefinition` has no page or parent fields. Extend that schema explicitly before trying to use `definition.page` or `definition.parent`.
- The group and button must have the same page. Layout still supplies their bounds; membership alone does not place them.
- Do not infer membership from the settings. prefix. The three string assignments establish it.

gui::Page: a tab ID, caption, and availability

A Page describes one tab. Page IDs are for membership and events; labels are for people reading the tab bar.

Return to edit steps: [tabs / N1 \(chart\)](#)

Declaration and examples:

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:113](#) | [symbol](#) | [Read in PDF \(p. 56\)](#)

Read-only supplier evidence: [@gui-boundary/examples/application.hpp:14](#) | [symbol](#) | [Read in PDF \(p. 57\)](#)

Synopsis

```
gui::Page{ id, label, [enabled, [visible]] }
```

Notation: Reading notation: square brackets mark optional trailing values, not literal C++ brackets. All four fields are filled below.

Numbered parameters

#	Parameter	Type	Meaning	Default	Filled example value
1	id	std::string	Unique nonempty page ID, used by active_page and WidgetSpec.page.	No member initializer; specify explicitly in this recipe.	"settings"
2	label	std::string	Human-readable tab caption.	No member initializer; specify explicitly in this recipe.	"Settings"
3	enabled	bool	Whether the page is available for selection.	true	true
4	visible	bool	Whether the page is available for display in the tab bar.	true	true

Filled example: declare two pages and select the initial page

```
view_.pages = {  
    gui::Page{  
        "entries",    // 1. id  
        "Entries",    // 2. label  
        true,         // 3. enabled  
        true          // 4. visible  
    },  
    gui::Page{  
        "settings",   // 1. id  
        "Settings",   // 2. label  
        true,         // 3. enabled  
        true          // 4. visible  
    }  
};  
view_.active_page = "entries"; // select by ID
```

- This is a proposed constructor fragment. The current application does not initialize pages.
- Declaring a page is only one edit: assign its widget membership, reserve `page_bar` geometry, and handle `PageEvent` as shown in the tab path.
- enabled and visible are independent. A page event for an unavailable page is rejected by normalization.

target_sources: target, visibility, then source paths

This CMake command appends files to an existing target. It uses whitespace-separated arguments, not C++ commas.

Return to edit steps: [source-files / N2 \(chart\)](#) | [source-files / N3 \(chart\)](#) | [source-files / N4 \(chart\)](#) | [source-files / N5 \(chart\)](#)

Declaration and examples:

Source: [gui/CMakeLists.txt:125](#) | [symbol](#)

Source: [gui/CMakeLists.txt:104](#) | [Read in PDF \(p. 7\)](#)

Synopsis

```
target_sources(TARGET PRIVATE SOURCE_PATH [MORE_SOURCE_PATHS...])
```

Notation: Reading notation: TARGET and SOURCE_PATH are placeholders; square brackets mean additional optional paths. PRIVATE is the literal keyword used by this recipe.

Numbered parameters

#	Parameter	Type	Meaning	Default	Filled example value
1	TARGET	existing CMake target name	The target whose implementation will compile this file.	Required for this recipe	foundation_gui_application
2	PRIVATE	literal scope keyword	Adds implementation sources to this target without publishing them as consumer interface sources.	Required scope keyword	PRIVATE
3+	SOURCE_PATH	one or more paths	Paths relative to this CMake directory in this example. Each additional file is a separate argument.	At least one path in this recipe	shared/custom_widget.cpp

Filled example: add one shared GUI source file

```
# In gui/CMakeLists.txt, after the target is declared:
target_sources(
    foundation_gui_application # 1. existing target
    PRIVATE                  # 2. scope
    shared/custom_widget.cpp # 3. source path
)
```

- This is an alternative to adding the file to the existing add_library source list; do not add the same file using both recipes.
- The example assumes the new file exists at gui/shared/custom_widget.cpp. Root CMakeLists.txt uses paths relative to the project root instead.
- An extra method in an already compiled .cpp needs no target_sources call. The documentation generator never runs this example or CMake.

Store::update: stable record ID and replacement text

Replace text on an existing record while retaining its ID. The bool distinguishes successful replacement from a missing record when the replacement text is valid.

Return to edit steps: [core-operation](#) / [N4](#) (chart)

Declaration and examples:

Source: [include/foundation/store.hpp:32](#) | [symbol](#) | [Read in PDF](#) (p. 18)

Source: [include/foundation/store.hpp:12](#) | [Read in PDF](#) (p. 121)

Source: [include/foundation/store.hpp:28](#) | [symbol](#) | [Read in PDF](#) (p. 18)

Source: [src/store.cpp:47](#) | [symbol](#) | [Read in PDF](#) (p. 78)

Synopsis

```
bool Store::update(RecordId id, std::string_view text)
```

Notation: This is the existing function signature. Unlike the aggregate examples, parentheses contain two function arguments; neither argument has a default.

Numbered parameters

#	Parameter	Type	Meaning	Default	Filled example value
1	id	foundation::RecordId (std::uint64_t)	Existing stable record identity, such as the ID returned by add. It is not a row index.	Required	id
2	text	std::string_view	Replacement text: 1 through 256 printable ASCII bytes. The Store prepares an owning string for successful replacement.	Required	"Revised text"

Filled example: update a record using the ID returned by add

```
foundation::Store store;
const foundation::RecordId id = store.add("Original text");

const bool changed = store.update(
    id,          // 1. stable ID returned by add
    "Revised text" // 2. replacement text
);
// Here changed is true, and get(id) still uses the same ID.
```

- This illustrates the existing API with a local Store; it is not the proposed GUI selection handler. Resolve the selected record string to its actual RecordId before calling update in that handler.
- Return true means the existing record text was replaced. Return false means no record has that ID, after the replacement text has passed validation.
- Invalid replacement text throws std::invalid_argument before the missing-ID lookup. A nonexistent ID combined with invalid text therefore throws rather than returning false.
- The implementation prepares a replacement string before swapping it into stored state. A failed operation preserves the collection and IDs; allocation or component failures can also propagate.
- Do not implement editing by erase followed by add: that creates a different ID. get and snapshot return copies, so modifying their returned text does not mutate Store.
- Callers serialize access to each Store instance, as the existing public contract requires.

gui::KeyBinding: key, button target, and modifiers

A shared shortcut targets a current button WidgetKey. Normalization converts the shortcut to that button's Activate event.

Return to edit steps: [commands / N5 \(chart\)](#)

Declaration and examples:

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:119](#) | [symbol](#) | [Read in PDF \(p. 89\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:118](#) | [Read in PDF \(p. 90\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:19](#) | [symbol](#) | [Read in PDF \(p. 51\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:321](#) | [symbol](#) | [Read in PDF \(p. 43\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:322](#) | [symbol](#) | [Read in PDF \(p. 43\)](#)

Read-only supplier evidence: [@gui-boundary/include/gui/contract.hpp:377](#) | [symbol](#) | [Read in PDF \(p. 91\)](#)

Synopsis

```
gui::KeyBinding{ key, target, [control, [shift, [alt]]] }
```

Notation: Reading notation: square brackets mark optional trailing modifier fields. target is a nested WidgetKey aggregate with its own id and generation.

Numbered parameters

#	Parameter	Type	Meaning	Default	Filled example value
1	key	gui::ShortcutKey	Supported key: escape, enter, or f1 through f12.	gui::ShortcutKey::escape	gui::ShortcutKey::f2
2	target	gui::WidgetKey	The exact identity of a current button: widget ID plus matching generation.	Empty ID and generation 1 until a target is supplied; an empty target is invalid for a binding.	gui::WidgetKey{"entries.update", 1}
3	control	bool	Whether the Control modifier must match this binding.	false	false
4	shift	bool	Whether the Shift modifier must match this binding.	false	false
5	alt	bool	Whether the Alt modifier must match this binding.	false	false

Filled example: F2 activates the proposed Update button after it exists

```
view_.key_bindings.push_back(gui::KeyBinding{
    gui::ShortcutKey::f2,    // 1. key
    gui::WidgetKey{         // 2. target
        "entries.update",   //    current button ID
        1                   //    matching current generation
    },
    false,                  // 3. control
    false,                  // 4. shift
    false                   // 5. alt
});
```

- entries.update is a proposed button ID, not a control currently present in the application. Declare/create that button and route its Activate event before adding this binding.
- The target must exist in the same snapshot and have Kind::button. A menu option ID is not a shortcut target.
- Generation 1 matches newly declared ordinary widgets in the current example. If the widget identity changes, use its actual current generation.
- The combination of key, control, shift and alt must be unique. false means that modifier is not pressed; it does not mean ignore the modifier.
- This pinned enum has no arbitrary letter-key values. Disabled or unavailable button input still follows normal event normalization.